Bazele inteligenței artificiale

Laborator – Grigore Ionuț

Laboratory contents

1. Introduction to Python
2. Linear models
3. Nonparametric models
4. Decision trees. Ensemble methods
5. Dimensionality reduction → **today's lab**
6. Clustering
7. Test 1
8. Introduction to PyTorch
9. Multilayer perceptrons
10. Convolutional neural networks
11. Recurrent neural networks
12. Attention mechanisms
13. Test 2
14. Recoveries

1:07:08

Dimensionality Reduction | ML-005 Lecture 14 | Stanford University | Andrew Ng

7,6 K de vizionări • acum 7 ani

M Machine Learning and AI

Contents: Motivation 1 - Data Compression, Motivation 2 - Visualization, **Principal Component Analysis** - Problem Formulation, ..

Content

- The curse of dimensionality - intro
- Principal component analysis (PCA)
- Locally linear embedding (LLE)
- t-Distributed stochastic neighbor embedding (t-SNE)

1) The curse of dimensionality

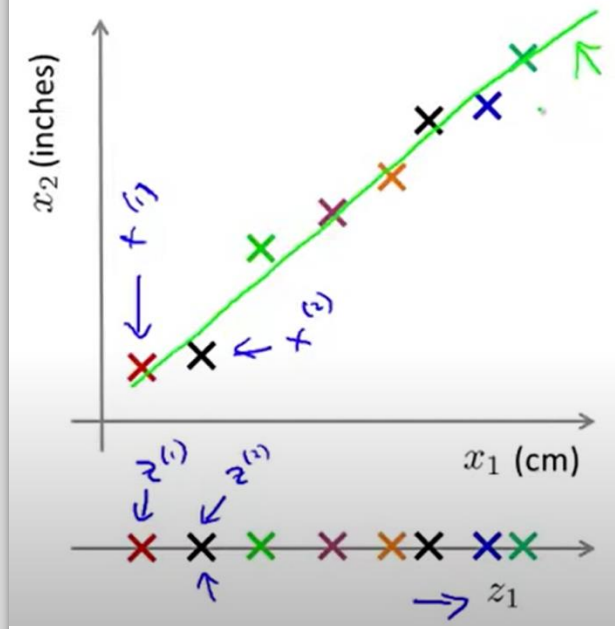
- many machine learning problems involve thousands or even millions of features for each training instance

- all these features
  - make training extremely slow
  - make it much harder to find a good solution

the curse of dimensionality !

Instance ID	Feature 1	Feature 2	Feature 3	...	Feature 1,000,000
1	7.1	5.4	2.1	...	1.3
2	6.8	4.9	2.5	...	2.0
3	6.5	5.1	2.3	...	1.8
...	...	...	...	...	...
1,000,000	5.4	3.9	1.5	...	3.2

Instance ID	Feature 1	Feature 2	Feature 3
1	7.1	5.4	2.1
2	6.8	4.9	2.5
3	6.5	5.1	2.3
...	...	...	...
1,000,000	5.4	3.9	1.5



Reduce data from 2D to 1D

$x^{(1)} \in \mathbb{R}^2 \rightarrow z^{(1)} \in \mathbb{R}$

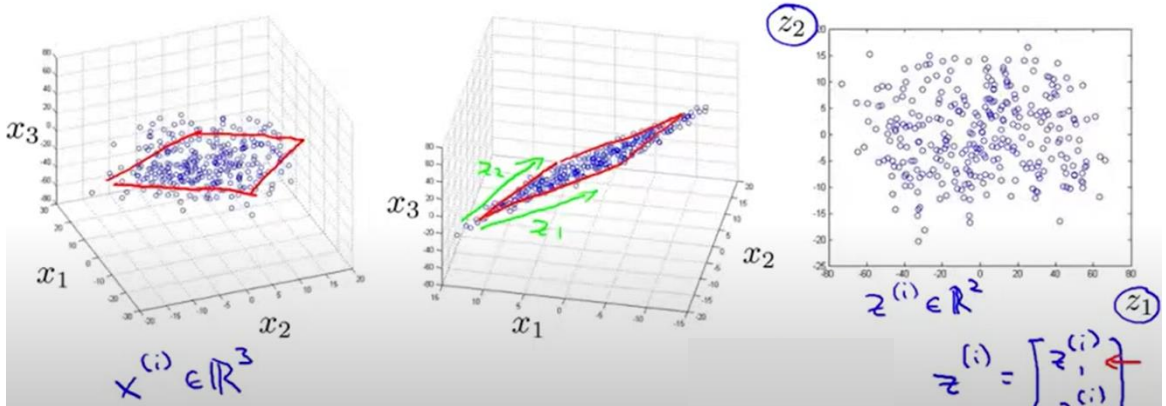
$x^{(2)} \in \mathbb{R}^2 \rightarrow z^{(2)} \in \mathbb{R}$

$\vdots$

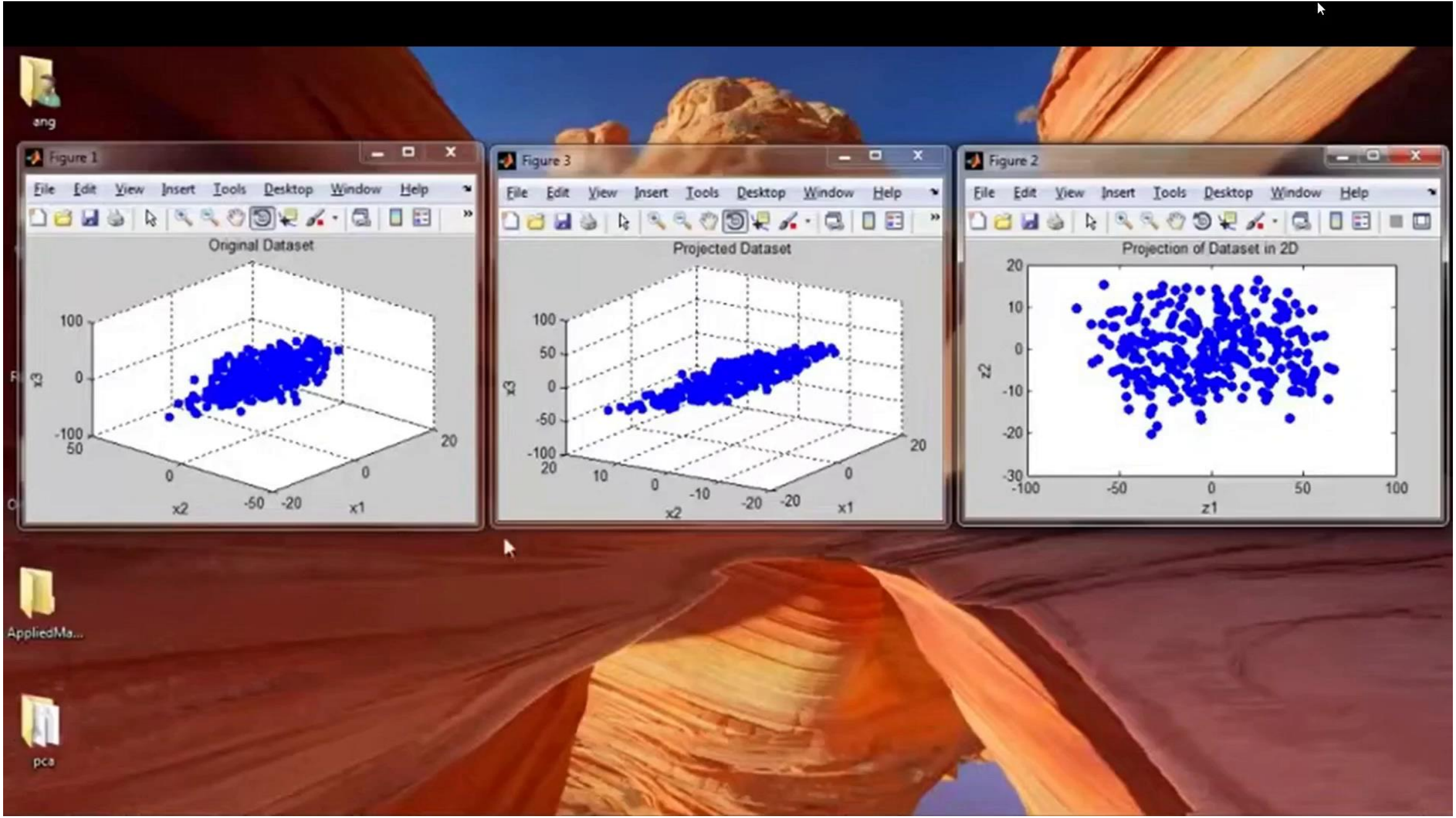
$x^{(m)} \in \mathbb{R}^2 \rightarrow z^{(m)} \in \mathbb{R}$

10000 → 100D

Reduce data from 3D to 2D



https://www.youtube.com/watch?v=pAwjiGkafbM


ang
Figure 1
File Edit View Insert Tools Desktop Window Help
Original Dataset
x3
x2
x1
Figure 3
File Edit View Insert Tools Desktop Window Help
Projected Dataset
x3
x2
x1
Figure 2
File Edit View Insert Tools Desktop Window Help
Projection of Dataset in 2D
z2
z1
AppliedMa...
pca

- in real world problems it's often possible to reduce the number of features considerably:

  - some information loss → a slightly worse performance
  - speed up training
  - data visualization → reducing the number of dimensions to 2 or 3 we can plot the data and gain some insights about the it

- Analogy:

  Like summarizing a 100-page book into 1 page — you lose details, but keep the essence.

Data Visualization

$x \in \mathbb{R}^{50}$ $\quad x^{(i)} \in \mathbb{R}^{50}$

Country	x_1 GDP (trillions of US$)	x_2 Per capita GDP (thousands of intl. $)	x_3 Human Develop-ment Index	x_4 Life expectancy	x_5 Poverty Index (Gini as percentage)	x_6 Mean household income (thousands of US$)	...
Canada	1.577	39.17	0.908	80.7	32.6	67.293	...
China	5.878	7.54	0.687	73	46.9	10.22	...
India	1.632	3.41	0.547	64.7	36.8	0.735	...
Russia	1.48	19.84	0.755	65.5	39.9	0.72	...
Singapore	0.223	56.69	0.866	80	42.5	67.1	...
USA	14.527	46.86	0.91	78.3	40.8	84.3	...
...	...	...	...	...	...	...	

Data Visualization

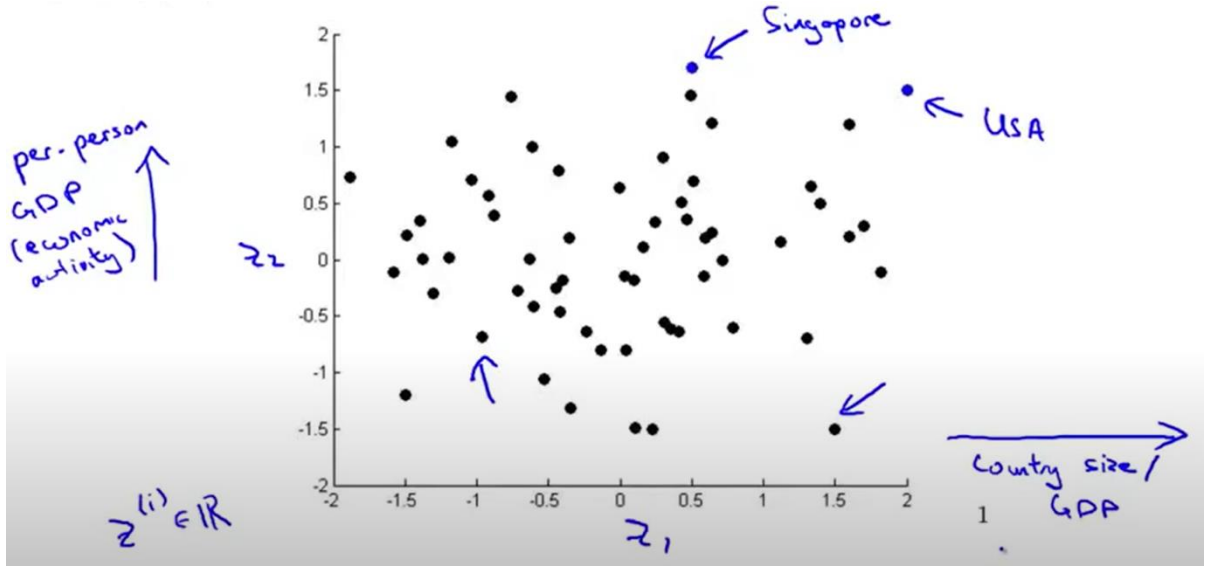
$z^{(i)} \in \mathbb{R}^2$

Country	z_1	z_2
Canada	1.6	1.2
China	1.7	0.3
India	1.6	0.2
Russia	1.4	0.5
Singapore	0.5	1.7
USA	2	1.5
...	...	...

Reduce data from 50D to 2D

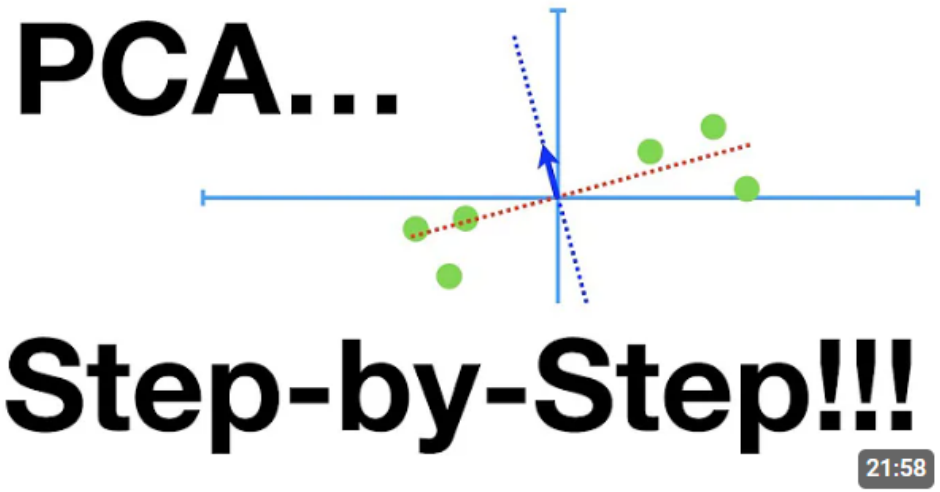
NOTE:

- the dimensionality reduction algorithm usually doesn't describe a particular meaning to these new features z_1 and z_2
- it's often up to us to figure out roughly what these features means



- we might find for example that the z_1 axis corresponds roughly to the overall country size or the overall economic activity (GDP) whereas the z_2 axis may correspond to per person economic activity

2) Principal component analysis (PCA)



StatQuest: Principal Component Analysis (PCA), Step-by-Step

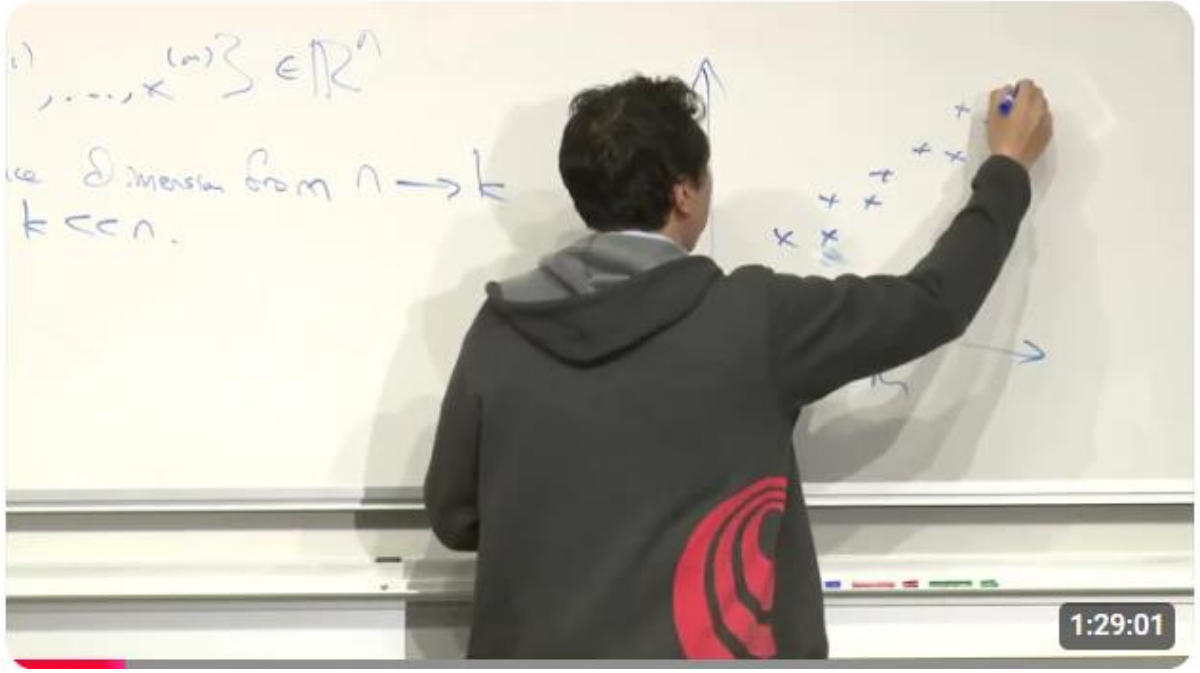
2,9 mil. de vizionări • acum 6 ani

StatQuest with Josh Starmer

Principal Component Analysis, is one of the most useful data analysis and machine learning methods out there. It can be used to ...

Subtitrări Colegiu

Capitole: 9 Awesome song and introduction | Conceptual motivation for PCA | PCA worked out for 2-...



PCA

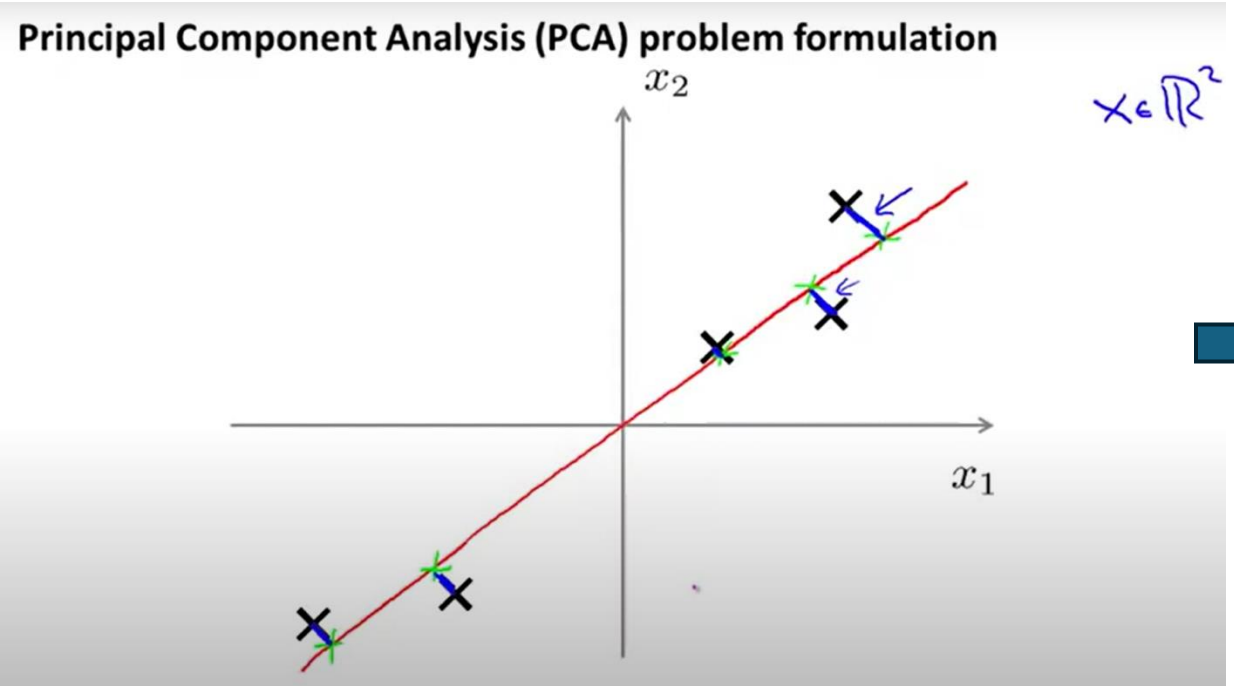
2,5 K de vizionări • acum 1 an

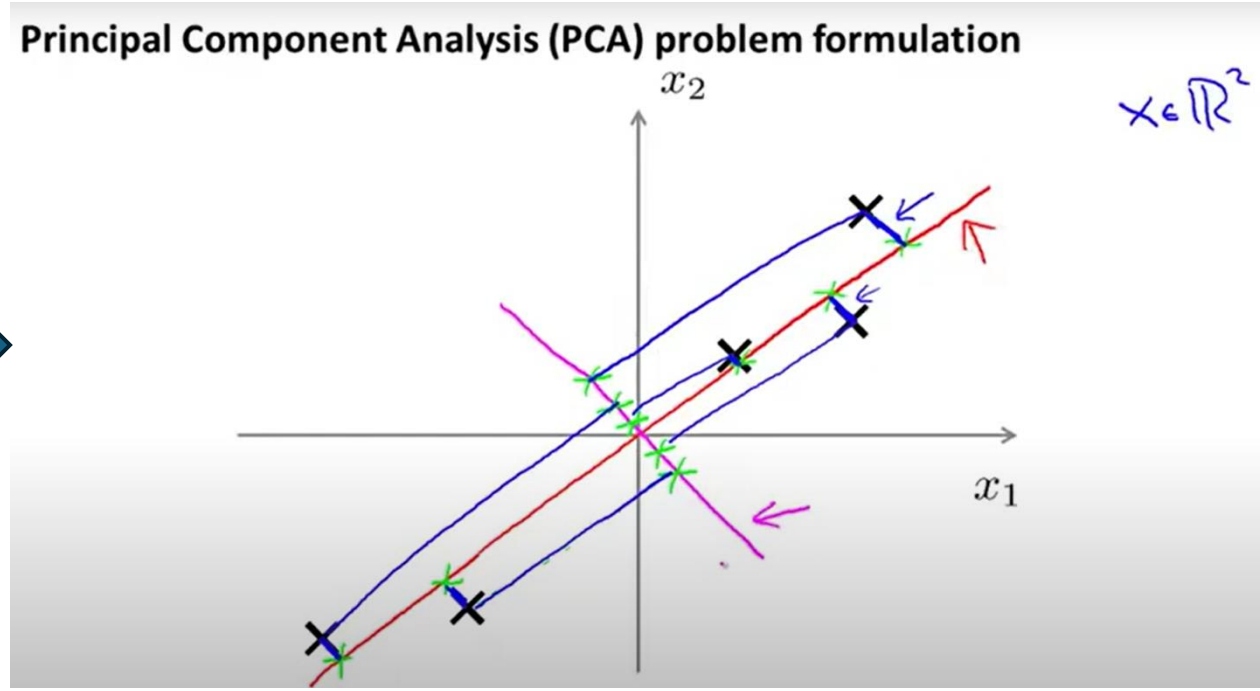
Ran Huang

CS229-2018 **Principal Component Analysis**, the missing episode according to the official source.

GENERAL IDEA: identify the hyperplane that lies closest to the data and then projects the data onto it

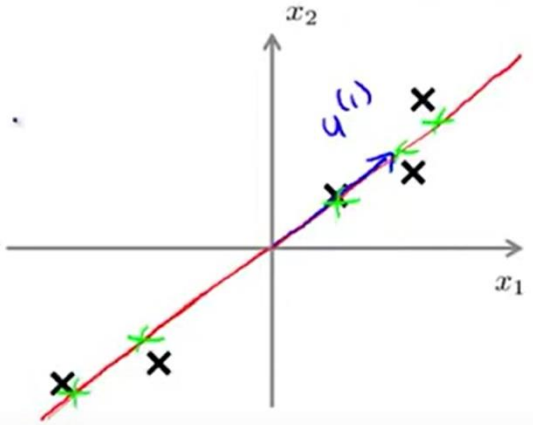
NOTE: before we can project the training set onto a lower-dimensional hyperplane, we first need to choose **THE RIGHT HYPERPLANE** !



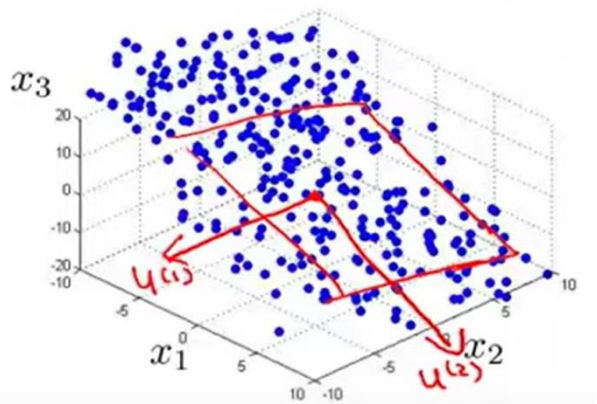


red line →

magenta line →



Reduce from 2-dimension to 1-dimension: Find a direction (a vector $u^{(1)} \in \mathbb{R}^n$) onto which to project the data so as to minimize the projection error.



Reduce from 3-dimension to 2-dimension: Find 2 vectors $u^{(1)}, u^{(2)}$ onto which to project the data, so as to minimize the projection error.

Reduce from n-dimension to k-dimension: Find k vectors $u^{(1)}, u^{(2)}, \ldots, u^{(k)}$ onto which to project the data, so as to minimize the projection error.

- PCA finds **new axes** (called **principal components**) that capture the **maximum variance** in the data

How it works:

- The **first principal component** points in the direction of **highest variance**.
- Each subsequent component is **orthogonal** (perpendicular) to the previous ones and captures the **next highest variance**.
- These components form a new coordinate system that better reveals the structure of the data.

Analogy: Imagine rotating the coordinate system to "see" the data from its most informative angle.

Fun fact: PCA was invented over 100 years ago — still one of the most used tools in AI today!

Dimensionality reduction (Pearson's words)

(1) IN many physical, statistical, and biological investigations it is desirable to represent a system of points in plane, three, or higher dimensioned space by the "best-fitting" straight line or plane. Analytically this

These two illustrations may suffice to show that the methods of this paper can be easily applied to numerical problems; the labour is not largely increased if we have a considerable number of points. It becomes more cumbersome if we have four, five, or more variables or characters which involve the

(1901)

Karl Pearson, 1901

[559]

LIII. *On Lines and Planes of Closest Fit to Systems of Points in Space.* By KARL PEARSON, *F.R.S., University College, London* *.

K. Pearson, "*On lines and planes of closest fit to systems of points in space*", The London, Edinburgh and Dublin Philosophical Magazine and Journal of Science, Sixth Series, 2, pp. 559-572 (1901)

(1857-1936)

PCA Steps

https://www.youtube.com/watch?v=pAwjiGkafbM

When you worry about it coz you don't understand

1)

Data preprocessing

Training set: $x^{(1)}, x^{(2)}, \ldots, x^{(m)}$

Preprocessing (feature scaling/mean normalization):

$$\mu_j = \frac{1}{m}\sum_{i=1}^{m} x_j^{(i)}$$

Replace each $x_j^{(i)}$ with $x_j - \mu_j$.

WHY ?

If different features on different scales (e.g., x_1 = size of house, x_2 = number of bedrooms), scale features to have comparable range of values.

https://www.youtube.com/watch?v=pAwjiGkafbM

Reduce data from n-dimensions to k-dimensions

2) Compute "covariance matrix":

$$\Sigma = \frac{1}{m}\sum_{i=1}^{n}(x^{(i)})(x^{(i)})^T$$

1 x n

n x 1

3) Compute "eigenvectors" of matrix Σ:

```
[U,S,V] = svd(Sigma);
```

n x n

4)
```
Ureduce = U(:,1:k);
```

5)
```
z = Ureduce'*x;
```

https://www.youtube.com/watch?v=pAwjiGkafbM

Principal Component Analysis (PCA) algorithm summary

After mean normalization (ensure every feature has zero mean) and optionally feature scaling:

$$\texttt{Sigma} = \frac{1}{m}\sum_{i=1}^{m}(x^{(i)})(x^{(i)})^T$$

$$X = \begin{bmatrix} — & x^{(1)T} & — \\ & \vdots & \\ — & x^{(m)T} & — \end{bmatrix}$$

$$\boxed{\texttt{Sigma = (1/m) * X' * X;}}$$

```
[U,S,V] = svd(Sigma);
Ureduce = U(:,1:k);
z = Ureduce'*x;
```

Principal Component Analysis (PCA) algorithm summary

- alternative variant used in the code part -

- **Step 1**: After mean normalization (ensure every feature has zero mean) and optional feature scaling:

$$X_{\text{centered}} = X - \text{mean}(X)$$

- **Step 2**: Apply SVD directly on the centered data matrix:

$$[U, S, V^T] = \text{svd}(X_{\text{centered}})$$

- **Step 3**: Select the top k principal components from V (right singular vectors):

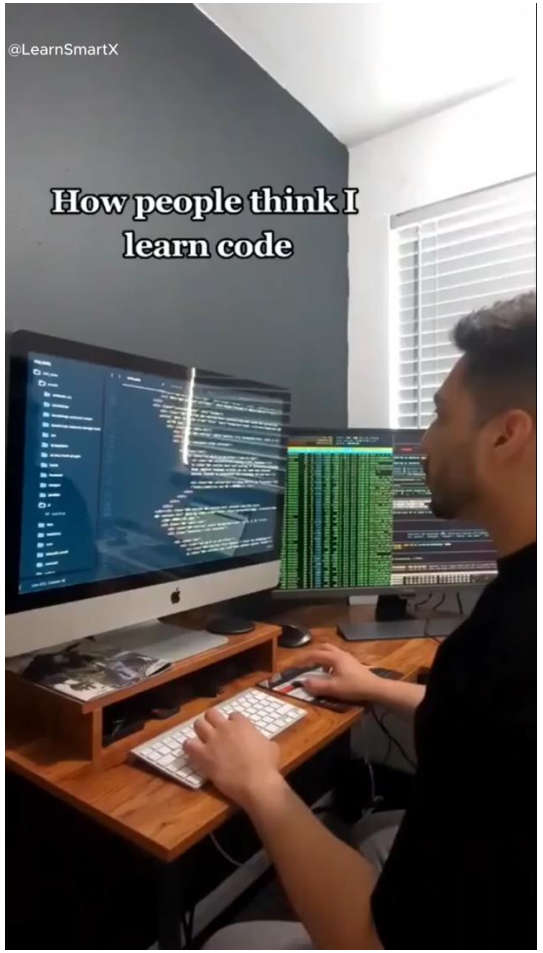
$$\text{V_reduce} = V[:, : k]$$

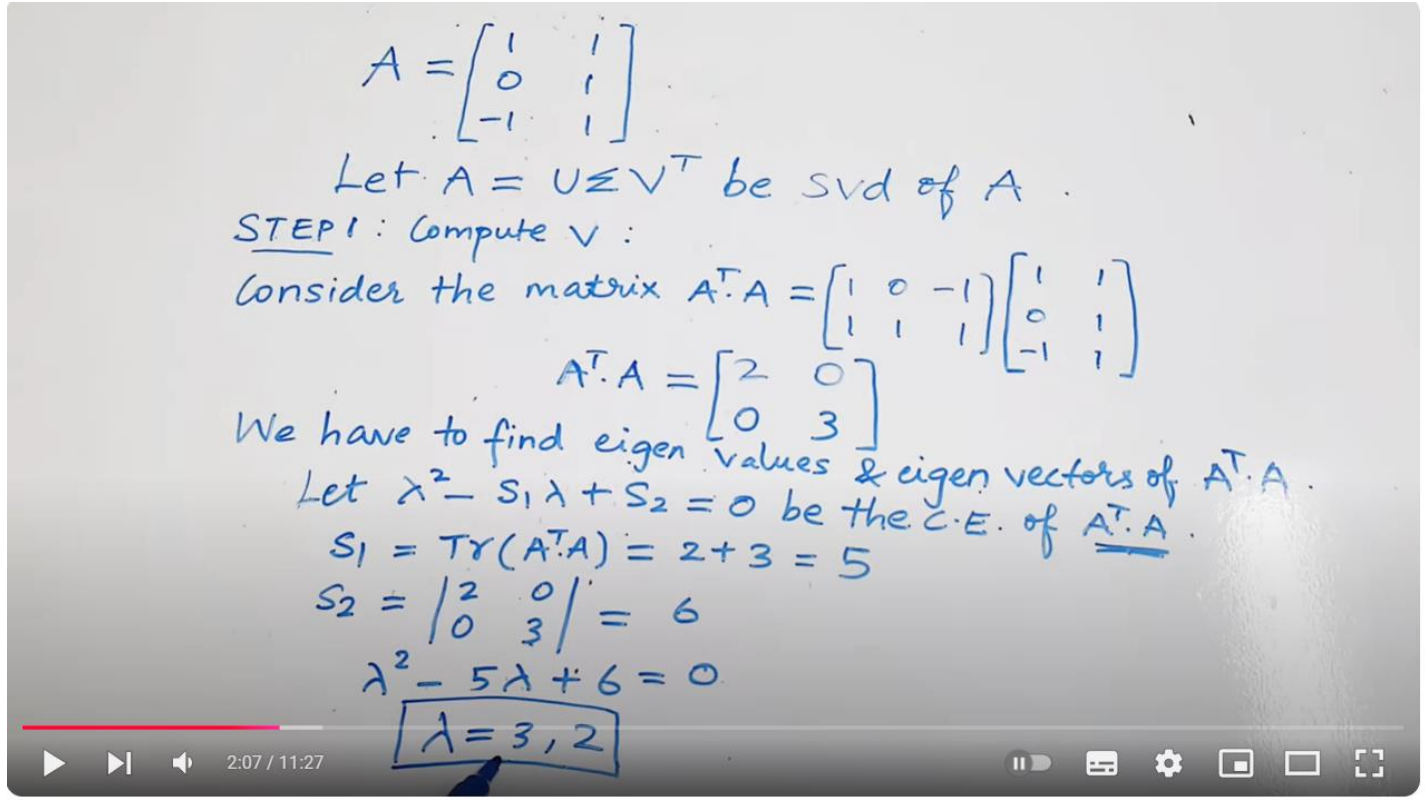
- **Step 4**: Project the data onto the reduced principal components:

$$Z = X_{\text{centered}} \times \text{V_reduce}$$

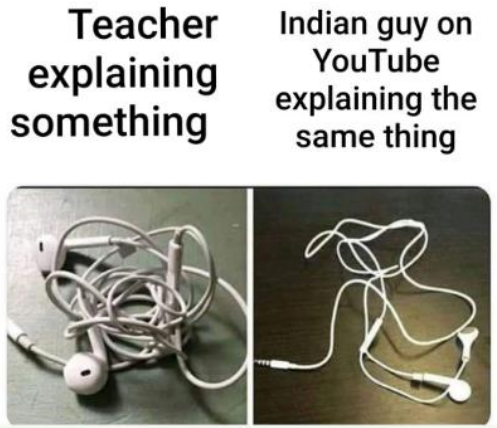
What is SVD?

https://medium.com/intuition/singular-value-decomposition-svd-working-example-c2b6135673b5





Singular Value Decomposition (SVD) Problem | Full Explanation



What SVD Really Does (Intuition)

Core idea

SVD (Singular Value Decomposition) takes any matrix and **breaks it into simpler, interpretable parts** — it shows *how data can be stretched, rotated, and scaled* in space.

Mathematically

$$X = U \Sigma V^T$$

Intuitive meaning

- **V^T** — directions of the **input space** (like the axes where data has structure)
- **Σ** — tells **how much each direction matters** (the amount of stretching)
- **U** — gives the **coordinates of data** after that stretching

In simple words

*SVD finds the **most important directions** in your data and tells you **how much information** each direction carries.*

DINOv3

Oriane Siméoni* Huy V. Vo* Maximilian Seitzer* Federico Baldassarre* Maxime Oquab*
Cijo Jose Vasil Khalidov Marc Szafraniec Seungeun Yi Michaël Ramamonjisoa
Francisco Massa Daniel Haziza Luca Wehrstedt Jianyuan Wang
Timothée Darcet Théo Moutakanni Leonel Sentana Claire Roberts
Andrea Vedaldi Jamie Tolan John Brandt[1] Camille Couprie
Julien Mairal[2] Hervé Jégou Patrick Labatut Piotr Bojanowski

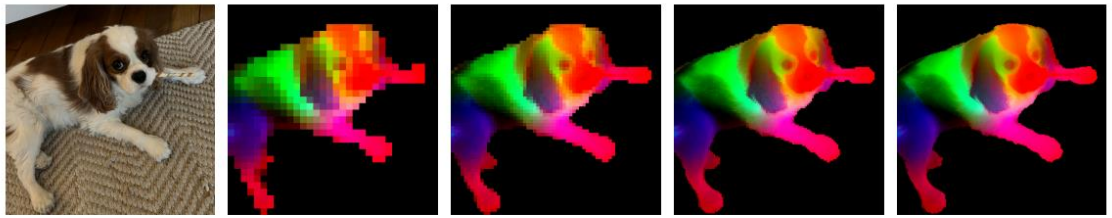
Meta AI Research [1]*WRI* [2]*Inria*

*corresponding authors: {osimeoni,huyvvo,seitzer,baldassarre,qas}@meta.com

13 Aug 2025

Abstract

Self-supervised learning holds the promise of eliminating the need for manual data annotation, enabling models to scale effortlessly to massive datasets and larger architectures. By not being tailored to specific tasks or domains, this training paradigm has the potential to



Input | 512 × 512 | 1024 × 1024 | 2048 × 2048 | 4096 × 4096

Figure 4: DINOv3 at very high resolution. We visualize dense features of DINOv3 by mapping the first three components of a PCA computed over the feature space to RGB. To focus the PCA on the subject, we mask the feature maps via background subtraction. With increasing resolution, DINOv3 produces crisp features that stay semantically meaningful. We visualize more PCAs in Sec. 6.1.1.

- DINOv3 is a new special **self-supervised model** that learns image features *without labels* released by Meta
- Each pixel in the image has a **feature vector** (hundreds of values) describing what the model "sees"
- These feature vectors live in a **high-dimensional space** — impossible to visualize directly
- PCA reduces these **high-dimensional pixel features** to just **3 components**
- The 3 principal components are mapped to **RGB colors**
- Similar colors = pixels that the network considers **semantically similar**
- This lets us **see** how the model internally organizes the image

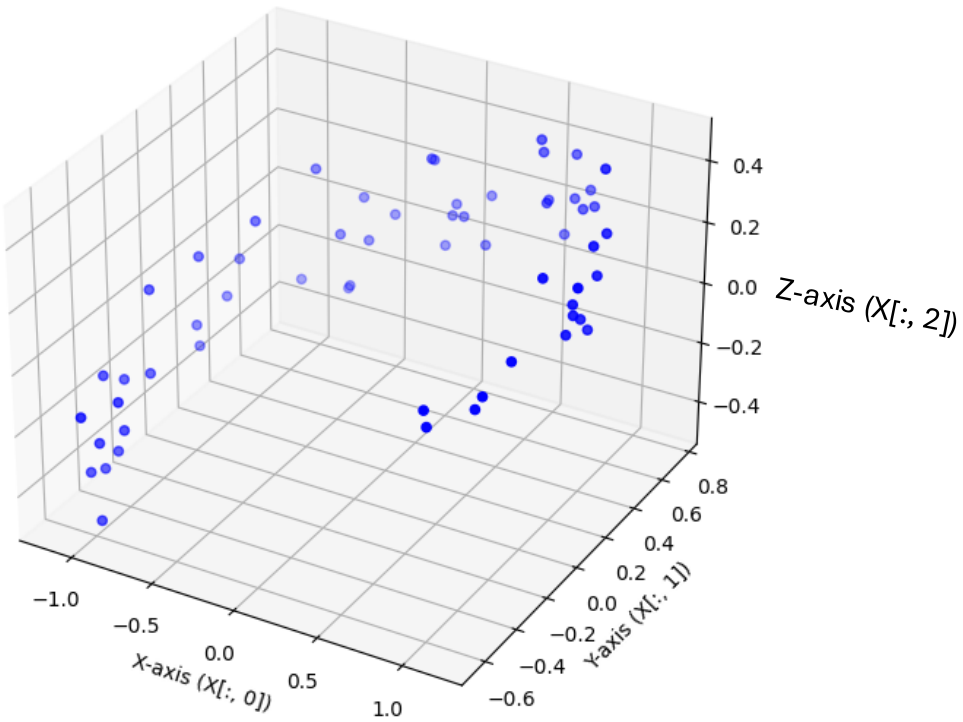
PCA makes the model's feature space visible —
it helps researchers **interpret, debug, and trust** how AI "understands" images.

- At my current job, we leverage **DINOv3** to segment teeth in smile images.
- Each pixel in the image has a **high-dimensional feature vector** that encodes semantic meaning.
- Using **PCA**, we can *project* these features into 3D RGB space to **visualize what DINO "understands"** about the image.
- **Left & Middle:** DINO correctly segments individual teeth with distinct PCA colors.
- **Right:** Remarkably, **tooth 16** (unlabeled in the ground truth) is still detected by DINO — **PCA visualization exposes that the model *internally recognizes* this tooth region**.

Code part

```python
import numpy as np

np.random.seed(4)
n = 60
w1, w2 = 0.1, 0.3
noise = 0.1

angles = np.random.rand(n) * 3 * np.pi / 2 - 0.5
X = np.empty((n, 3))
X[:, 0] = np.cos(angles) + np.sin(angles)/2 + noise * np.random.randn(n) / 2
X[:, 1] = np.sin(angles) * 0.7 + noise * np.random.randn(n) / 2
X[:, 2] = X[:, 0] * w1 + X[:, 1] * w2 + noise * np.random.randn(n)
```

Our 3D dataset:



feature scaling/mean normalization:

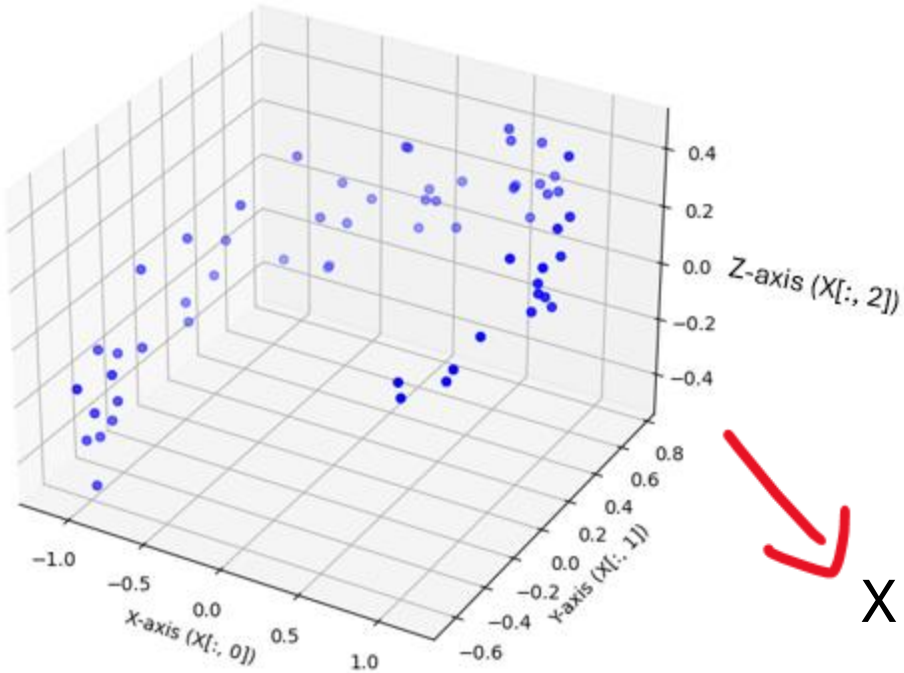
apply SVD (single value decomposition):

extract the 2 unit vectors that define the first 2 PCs :

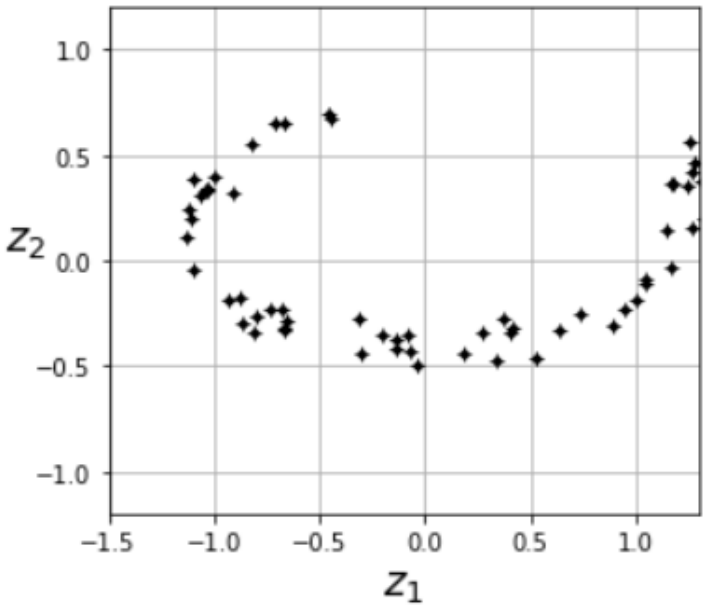
```
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)
c1 = Vt.T[:, 0]
c2 = Vt.T[:, 1]
```

obtain the reduced dataset:

```
W2 = Vt.T[:, :2]
X2D_using_svd = X_centered.dot(W2)
```



X

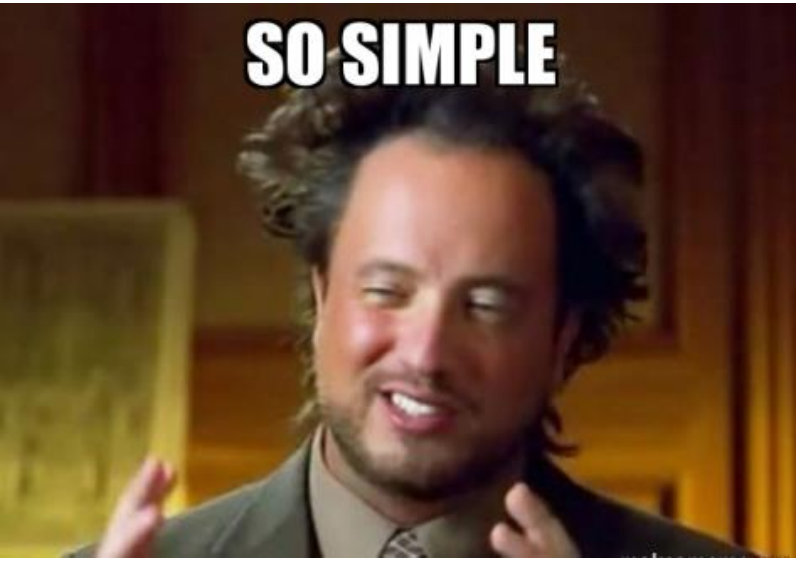


X2D_using_svd

...or in a simpler way...

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X2D = pca.fit_transform(X)
```

the same with previous X2D_using_svd



this is how we can see the components computed by PCA:

```
pca.components_
```

```
array([[-0.93636116, -0.29854881, -0.18465208],
       [ 0.34027485, -0.90119108, -0.2684542 ]])
```

- we can transform the data back to the original space, obtaining $\hat{X}$:

```
X3D_inv = pca.inverse_transform(X2D)
```

- of course, there was some loss of information during the projection step, so the recovered 3D points are not exactly equal to the original 3D points:

```
np.allclose(X3D_inv, X)
```

```
False
```

reconstruction error

```
np.mean(np.sum(np.square(X3D_inv - X), axis=1))
```

```
0.010170337792848549
```

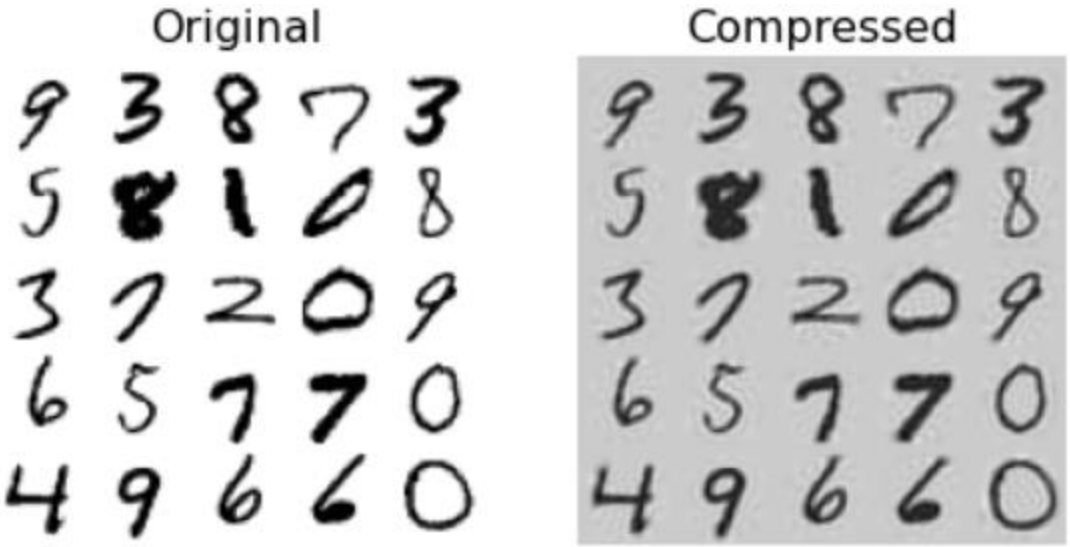
```python
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', version=1, as_frame=False)
mnist.target = mnist.target.astype(np.uint8)
```

```python
from sklearn.model_selection import train_test_split

X = mnist["data"]
y = mnist["target"]

X_train, X_test, y_train, y_test = train_test_split(X, y)
```

```python
pca = PCA(n_components=154)
X_reduced = pca.fit_transform(X_train)
X_recovered = pca.inverse_transform(X_reduced)
```

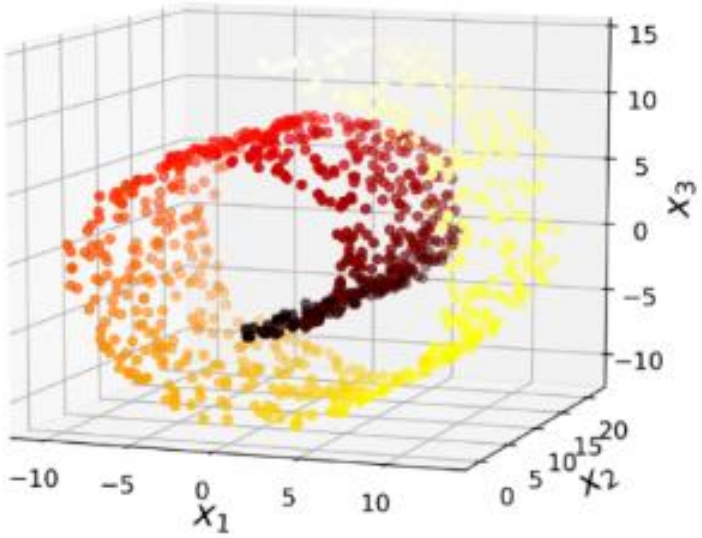
X_train

plot_digits() function from the lab

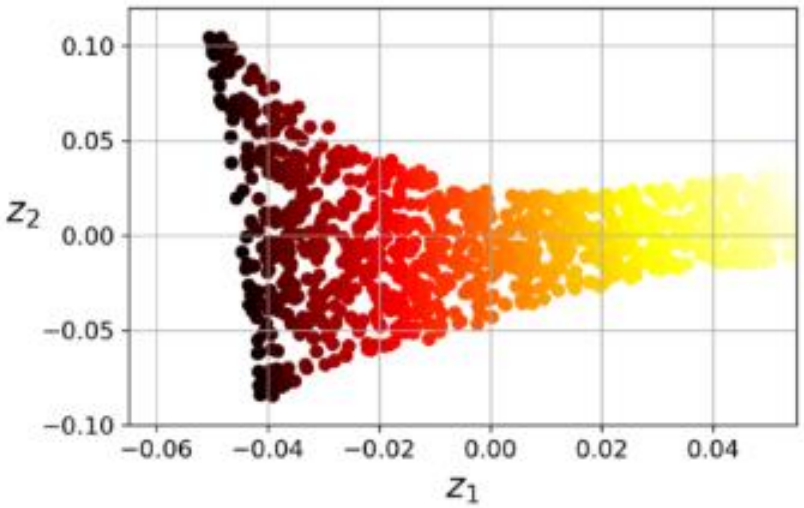


X_recovered

3) Locally linear embedding (LLE)



Swiss roll dataset



Unroleld Swiss roll

- it's a **manifold** learning algorithm → it's used for datasets like Swiss roll

- in a nutshell, LLE works by:
  1) measuring how each training instance linearly relates to it *closest neighbors*
  2) looking for a low-dimensional representation of the training set where these local relationships are best preserved

Locally Linear Embedding (LLE) Algorithm Summary

Don't worry about it ☺

- **Step 1: Normalize the data** (optional) to ensure consistency in scaling across features.
- **Step 2: For each data point $x^{(i)}$, find its k-nearest neighbors.** This step identifies the local neighborhood for each point.

$$\text{Neighbors}(x^{(i)}) = \text{k-nearest neighbors of } x^{(i)}$$

- **Step 3: Compute the weights** that best reconstruct each data point from its neighbors. For each point $x^{(i)}$, find weights w_{ij} that minimize the reconstruction error:

$$\text{minimize} \quad \|x^{(i)} - \sum_{j \in \text{Neighbors}(x^{(i)})} w_{ij} x^{(j)}\|^2$$

  - Subject to: $\sum_j w_{ij} = 1$
  - This is typically solved by constrained least squares.

- **Step 4: Compute the embedding** by minimizing the cost function based on the weights w_{ij}. The goal is to find a lower-dimensional representation Y that preserves these local relationships:

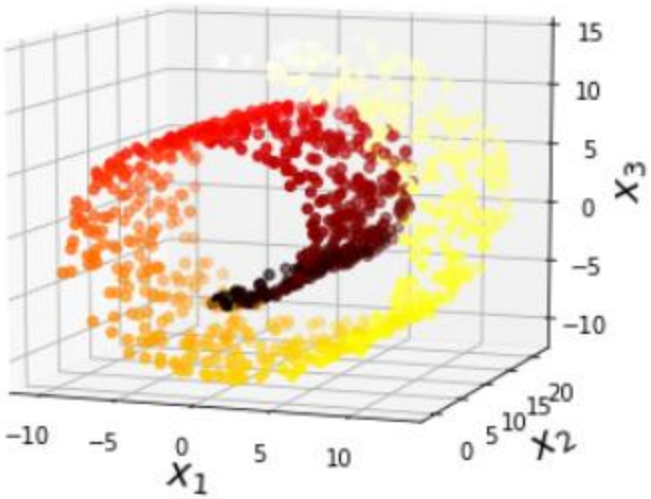
$$\text{minimize} \quad \sum_i \|y^{(i)} - \sum_{j \in \text{Neighbors}(x^{(i)})} w_{ij} y^{(j)}\|^2$$

- **Step 5: Solve for Y** using an eigenvector decomposition. The coordinates Y are obtained by selecting the eigenvectors corresponding to the smallest non-zero eigenvalues of the cost function matrix.
- **Output**: The reduced representation Y of the data in a lower-dimensional space.

Code part

```python
from sklearn.datasets import make_swiss_roll

X, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=41)
```

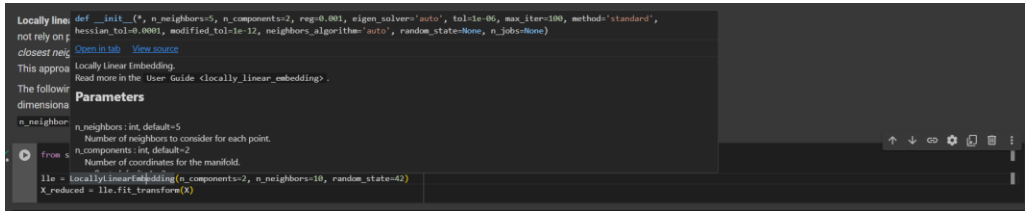
```python
axes = [-11.5, 14, -2, 23, -12, 15]

fig = plt.figure(figsize=(6, 5))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X[:, 0], X[:, 1], X[:, 2], c=t, cmap=plt.cm.hot)
ax.view_init(10, -70)
ax.set_xlabel("$x_1$", fontsize=18)
ax.set_ylabel("$x_2$", fontsize=18)
ax.set_zlabel("$x_3$", fontsize=18)
ax.set_xlim(axes[0:2])
ax.set_ylim(axes[2:4])
ax.set_zlim(axes[4:6])

plt.show()
```



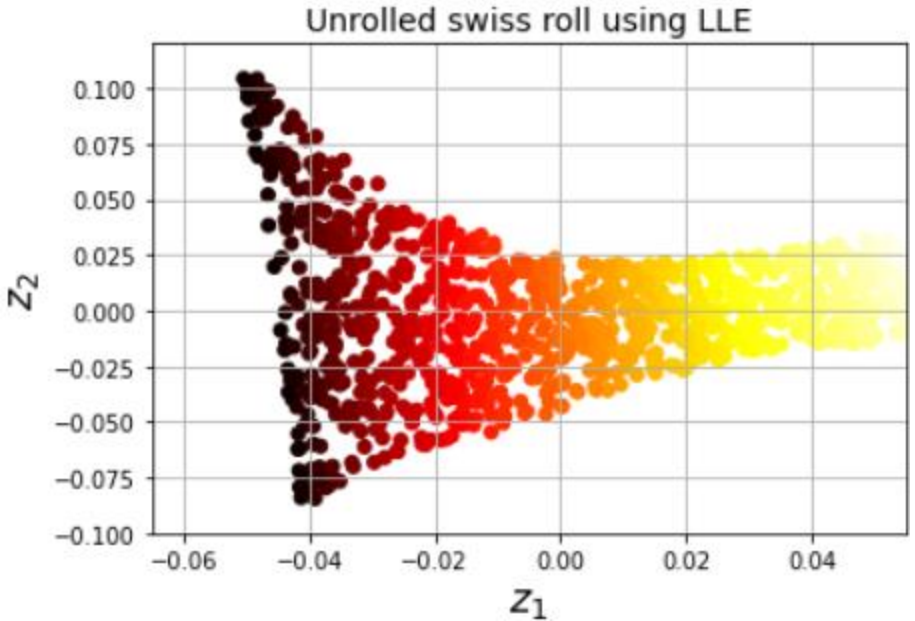
```python
from sklearn.manifold import LocallyLinearEmbedding

lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10, random_state=42)
X_reduced = lle.fit_transform(X)
```

```python
plt.title("Unrolled swiss roll using LLE", fontsize=14)
plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=t, cmap=plt.cm.hot)
plt.xlabel("$z_1$", fontsize=18)
plt.ylabel("$z_2$", fontsize=18)
plt.axis([-0.065, 0.055, -0.1, 0.12])
plt.grid(True)

plt.show()
```

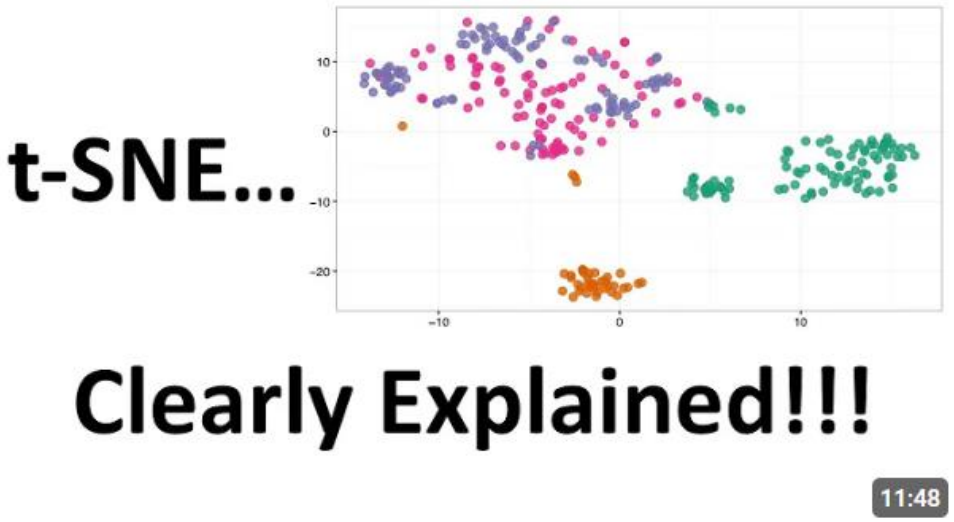
don't worry
about them ☺



KIND REMINDER



4) t-Distributed stochastic neighbor embedding (t-SNE)



StatQuest: t-SNE, Clearly Explained

471 K de vizionări • acum 7 ani

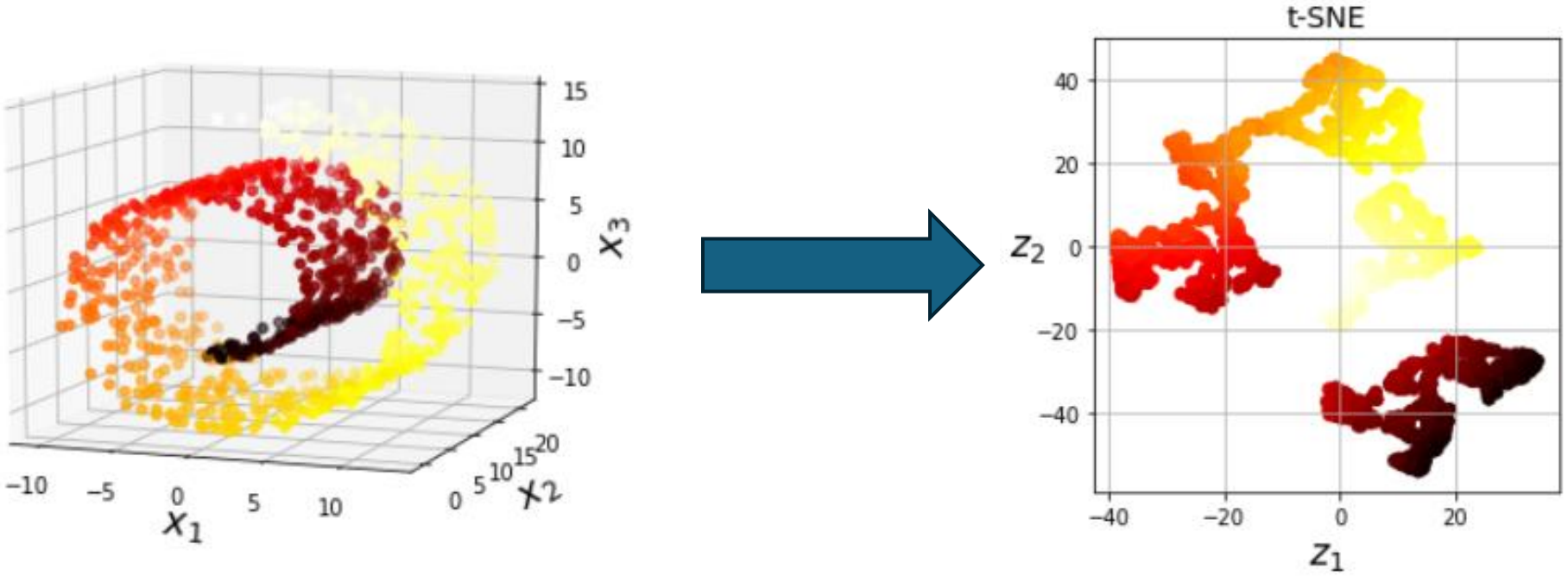
StatQuest with Josh Starmer

t-SNE is a popular method for making an easy to read graph from a complex dataset, but not many people know how it works.

Capitole: 7 Awesome song and introduction | Overview of what t-SNE does | Overview of how t-SNE works | Ste...

- t-SNE reduces dimensionality while trying to keep similar instances close and dissimilar instances apart

- it's very good for visualizing clusters of instances in high-dimensional space



t-SNE Algorithm Summary

- **Step 1: Compute Pairwise Similarities in the High-Dimensional Space**
  - For each pair of data points $x^{(i)}$ and $x^{(j)}$, calculate the similarity based on a Gaussian (normal) distribution centered at $x^{(i)}$. The similarity $p_{j|i}$ represents the probability that $x^{(j)}$ would be chosen as a neighbor of $x^{(i)}$.
  - Define $p_{j|i}$ as:

$$p_{j|i} = \frac{\exp(-\|x^{(i)} - x^{(j)}\|^2/2\sigma_i^2)}{\sum_{k\neq i}\exp(-\|x^{(i)} - x^{(k)}\|^2/2\sigma_i^2)}$$

  - Here, σ_i is a parameter that controls the width of the Gaussian for point $x^{(i)}$ and is chosen to maintain a desired "perplexity" (a parameter that influences the number of effective neighbors).
  - Symmetrize the probabilities:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

    where n is the number of data points.
- **Step 2: Define Pairwise Similarities in the Low-Dimensional Space**
  - Initialize a low-dimensional representation $y^{(i)}$ for each data point $x^{(i)}$, typically in 2D or 3D space.
  - Define the similarity q_{ij} between points $y^{(i)}$ and $y^{(j)}$ in the low-dimensional space based on a Student's t-distribution with one degree of freedom (which has heavy tails, helping to handle crowding issues):

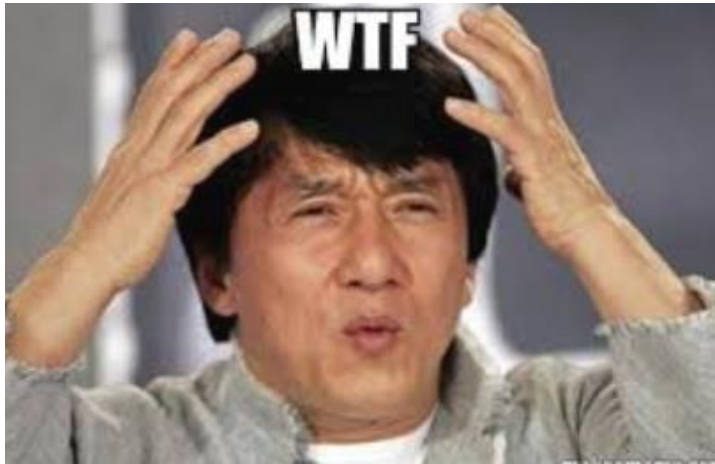
$$q_{ij} = \frac{(1 + \|y^{(i)} - y^{(j)}\|^2)^{-1}}{\sum_{k\neq l}(1 + \|y^{(k)} - y^{(l)}\|^2)^{-1}}$$

- **Step 3: Minimize the Kullback-Leibler Divergence**
  - The goal is to make q_{ij} in the low-dimensional space as similar as possible to p_{ij} from the high-dimensional space. This is done by minimizing the **Kullback-Leibler (KL) divergence** between the probability distributions P and Q:

$$\mathrm{KL}(P \parallel Q) = \sum_{i\neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

- **Step 4: Optimize the Low-Dimensional Embedding**
  - Use gradient descent to minimize the KL divergence with respect to the low-dimensional coordinates $y^{(i)}$. Update the positions of $y^{(i)}$ iteratively to reduce the KL divergence.
  - This step is computationally intensive, but it effectively adjusts the low-dimensional points so that their pairwise similarities q_{ij} approximate the high-dimensional similarities p_{ij}.
- **Step 5: Visualize the Results**
  - The final low-dimensional representations Y (usually in 2D or 3D) can now be visualized. Points that were close in high-dimensional space remain close in the low-dimensional embedding, and distinct clusters or structures in the data become visible.

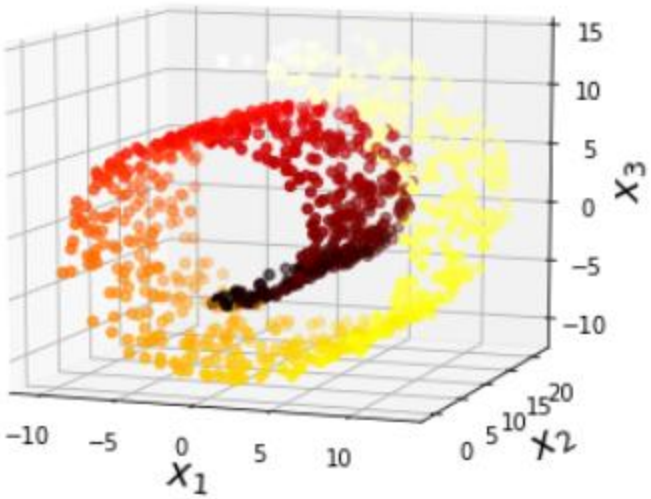
Again, don't worry about it ☺



Code part

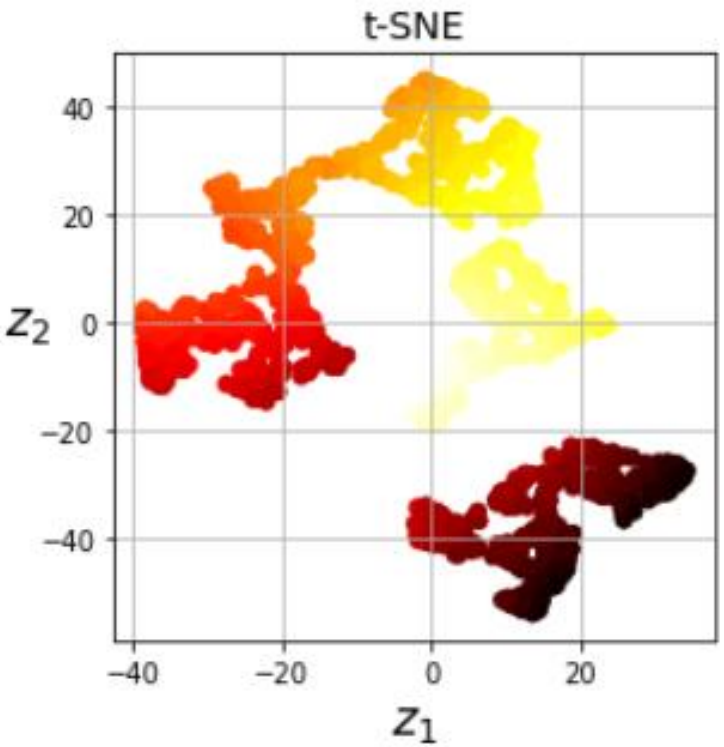
```python
from sklearn.datasets import make_swiss_roll

X, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=41)
```

```python
axes = [-11.5, 14, -2, 23, -12, 15]

fig = plt.figure(figsize=(6, 5))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X[:, 0], X[:, 1], X[:, 2], c=t, cmap=plt.cm.hot)
ax.view_init(10, -70)
ax.set_xlabel("$x_1$", fontsize=18)
ax.set_ylabel("$x_2$", fontsize=18)
ax.set_zlabel("$x_3$", fontsize=18)
ax.set_xlim(axes[0:2])
ax.set_ylim(axes[2:4])
ax.set_zlim(axes[4:6])

plt.show()
```



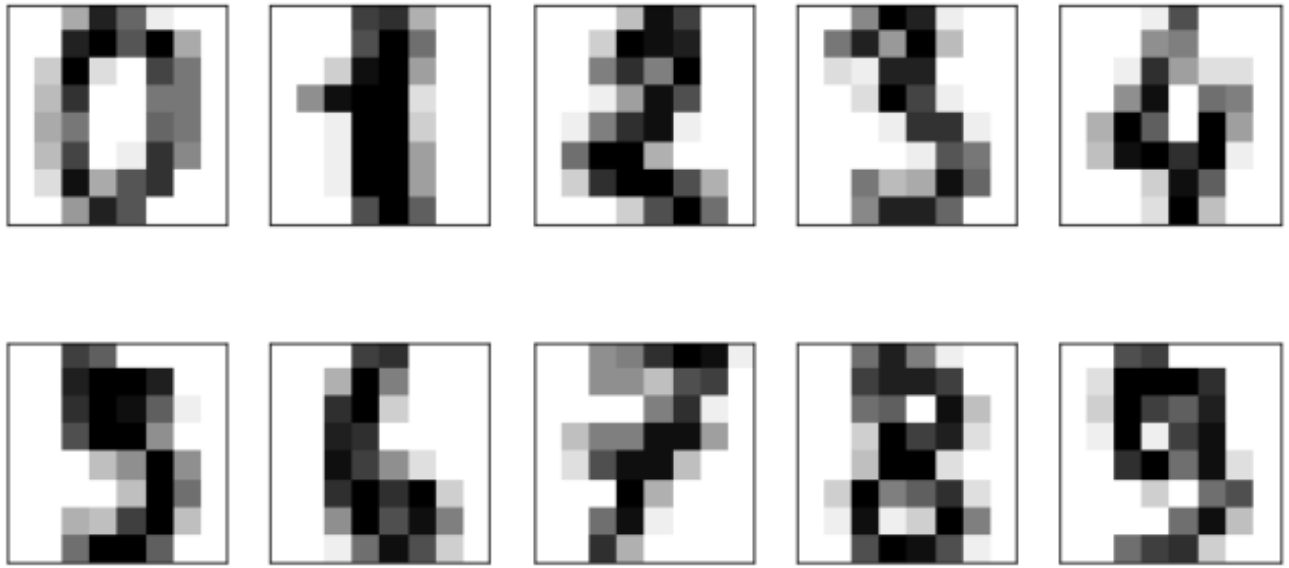
```python
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, random_state=42)
X_reduced_tsne = tsne.fit_transform(X)
```

```python
plt.figure(figsize=(4,4))

plt.title("t-SNE", fontsize=14)
plt.scatter(X_reduced_tsne[:, 0], X_reduced_tsne[:, 1], c=t, cmap=plt.cm.hot)
plt.xlabel("$z_1$", fontsize=18)
plt.ylabel("$z_2$", fontsize=18, rotation=0)
plt.grid(True)

plt.show()
```



```python
from sklearn.datasets import load_digits
digits = load_digits()
```

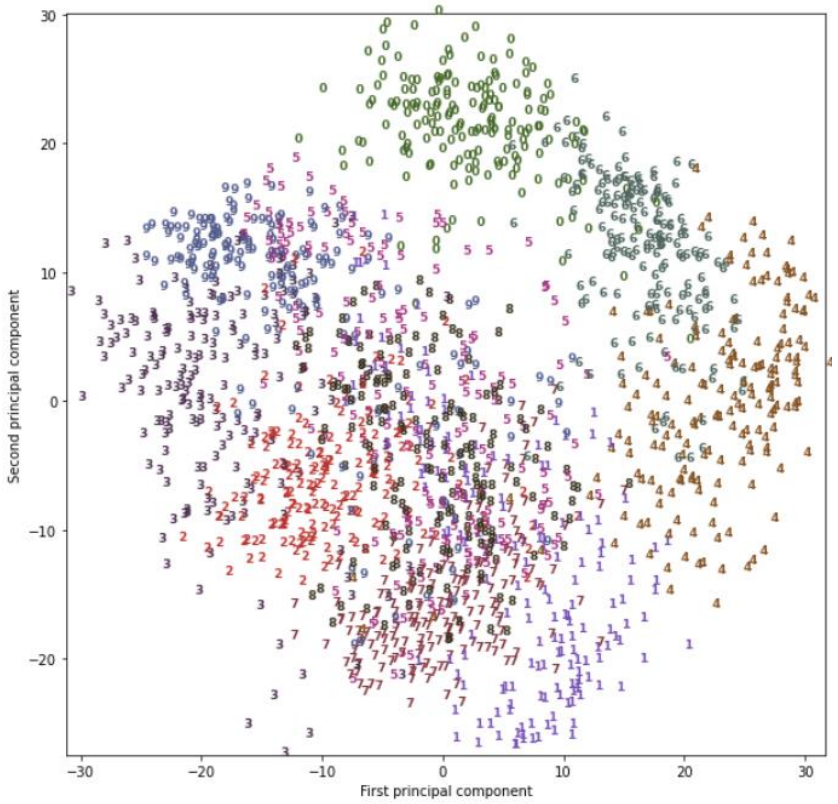
```python
fig, axes = plt.subplots(2, 5, figsize=(10, 5),
                         subplot_kw={'xticks':(), 'yticks': ()})
for ax, img in zip(axes.ravel(), digits.images):
    ax.imshow(img, cmap=mpl.cm.binary)
```



```python
# build a PCA model
pca = PCA(n_components=2)
pca.fit(digits.data)
# transform the digits data onto the first two principal components
digits_pca = pca.transform(digits.data)
```

PCA

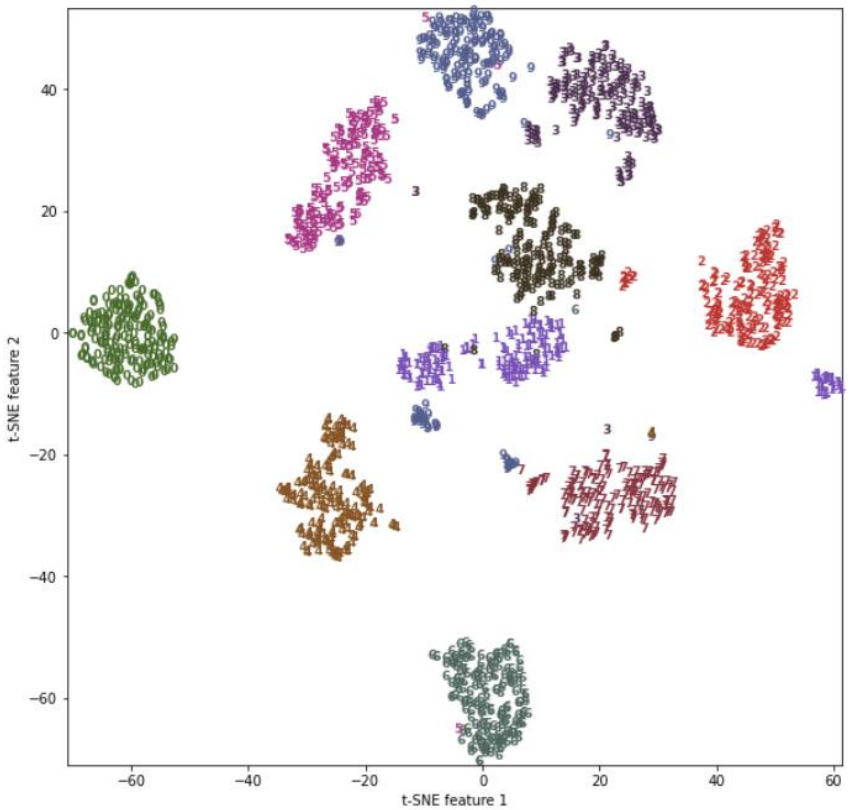
```python
colors = ["#476A2A", "#7851B8", "#BD3430", "#4A2D4E", "#875525",
          "#A83683", "#4E655E", "#853541", "#3A3120", "#535D8E"]
plt.figure(figsize=(10, 10))
plt.xlim(digits_pca[:, 0].min(), digits_pca[:, 0].max())
plt.ylim(digits_pca[:, 1].min(), digits_pca[:, 1].max())
for i in range(len(digits.data)):
    # actually plot the digits as text instead of using scatter
    plt.text(digits_pca[i, 0], digits_pca[i, 1], str(digits.target[i]),
             color = colors[digits.target[i]],
             fontdict={'weight': 'bold', 'size': 9})
plt.xlabel("First principal component")
plt.ylabel("Second principal component")
plt.show()
```




```python
from sklearn.manifold import TSNE
tsne = TSNE(random_state=42)
digits_tsne = tsne.fit_transform(digits.data)
```

t-SNE

```python
plt.figure(figsize=(10, 10))
plt.xlim(digits_tsne[:, 0].min(), digits_tsne[:, 0].max() + 1)
plt.ylim(digits_tsne[:, 1].min(), digits_tsne[:, 1].max() + 1)
for i in range(len(digits.data)):
    # actually plot the digits as text instead of using scatter
    plt.text(digits_tsne[i, 0], digits_tsne[i, 1], str(digits.target[i]),
             color = colors[digits.target[i]],
             fontdict={'weight': 'bold', 'size': 9})
plt.xlabel("t-SNE feature 1")
plt.ylabel("t-SNE feature 2")
plt.show()
```



Conclusion from the previous slide?

t-SNE reduces dimensionality while also helps for clustering !

Exercises

Laboratory 5 – Exercises

1. Use the `iris` dataset in order to reduce its dimensionality down to three dimensions by using the PCA algorithm. Print the principal components and the explained variance ratio and plot the 3D dataset. Also visualize the data reduced to three dimensions using the t-SNE method.

2. Use the `digits` dataset to perform a classification task using Linear Support Vector Machine (SVM). Set the parameter `C` to 5. Split the data such that the test split is 25% of the initial dataset. Compare the image classification task using the original number of features (64) to the reduced features (20) using the PCA algorithm. Compare:
   - ➢ the training time for both models;
   - ➢ the difference in accuracy score.

Solutions

1. Use the `iris` dataset in order to reduce its dimensionality down to three dimensions by using the PCA algorithm. Print the principal components and the explained variance ratio and plot the 3D dataset. Also visualize the data reduced to three dimensions using the t-SNE method.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris_dataset = load_iris() # Loading the Iris dataset
X, y = iris_dataset.data, iris_dataset.target # Splitting the dataset into features (X) and target (y)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0) # Splitting data into training and testing sets
```

```
from sklearn.decomposition import PCA

pca = PCA(n_components=3) # Initializing PCA with 3 components (reduce the data to 3 dimensions)
X3D = pca.fit_transform(X_train) # Fitting PCA to the data and transforming it to 3D
```

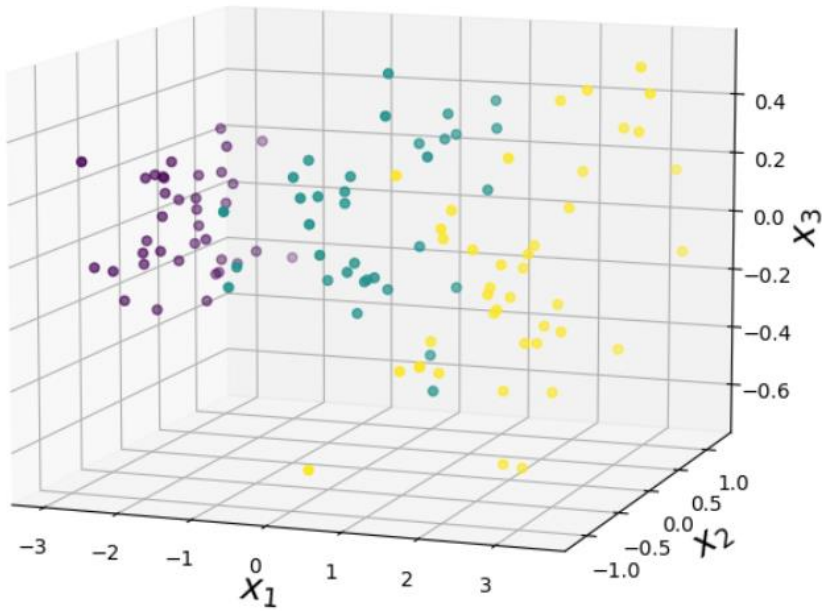
```
#Principal components
pca.components_
```

```
array([[ 0.37649644, -0.06637905,  0.85134571,  0.35924188],
       [ 0.6240207 ,  0.75538031, -0.18479376, -0.07648543],
       [ 0.60667794, -0.57674603, -0.08522779, -0.54040922]])
```

```
#Explained variance ratio
pca.explained_variance_ratio_
```

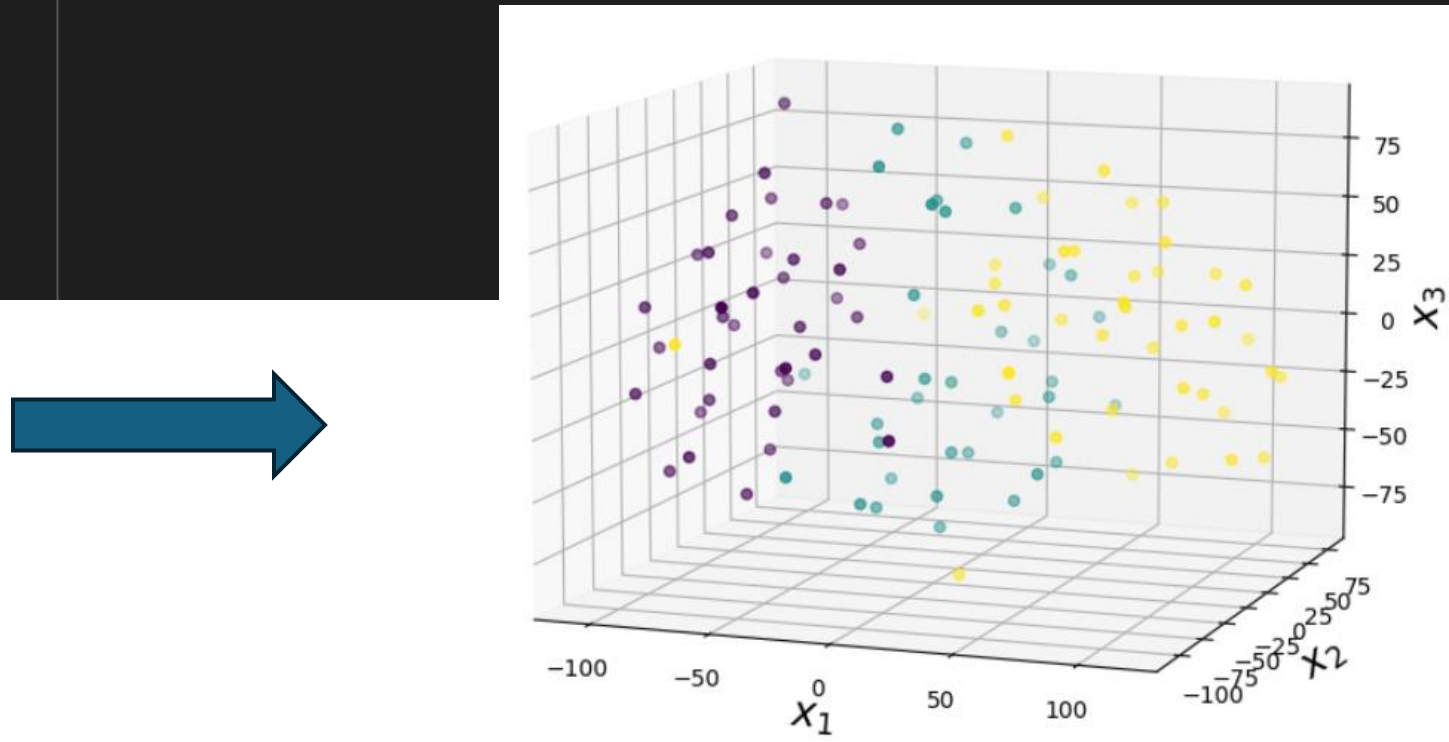
```
array([0.92743115, 0.05245721, 0.01557205])
```

```python
#3D plot for PCA
import matplotlib.pyplot as plt

# Setting the axis limits for the 3D plot (min and max for each dimension)
axes = [(X3D[:,0]).min(),(X3D[:,0]).max(),(X3D[:,1]).min(),(X3D[:,1]).max(),(X3D[:,2]).min(),(X3D[:,2]).max()]

fig = plt.figure(figsize=(8, 6), constrained_layout=True)
ax = fig.add_subplot(111, projection='3d')

# Plotting the data points, coloring them according to the target classes
ax.scatter(X3D[:, 0], X3D[:, 1], X3D[:, 2], c=y_train, cmap=plt.cm.viridis)
ax.view_init(10, -70)
ax.set_xlabel("$x_1$", fontsize=18)
ax.set_ylabel("$x_2$", fontsize=18)
ax.set_zlabel("$x_3$", fontsize=18)
ax.set_xlim(axes[0:2])
ax.set_ylim(axes[2:4])
ax.set_zlim(axes[4:6])

plt.show()
```



```python
#t-SNE
from sklearn.manifold import TSNE

# Initializing t-SNE to reduce dimensionality to 3 dimensions (non-linear method)
tsne = TSNE(n_components=3, random_state=42)
# Fitting t-SNE to the data and transforming it to 3D
X_reduced_tsne = tsne.fit_transform(X_train)
```

```python
#t-SNE 3D plot

# Setting the axis limits for the 3D plot (min and max for each dimension)
axes = [(X_reduced_tsne[:,0]).min(),(X_reduced_tsne[:,0]).max(),(X_reduced_tsne[:,1]).min(),(X_reduced_tsne[:,1]).max(),(X_reduced_tsne[:,2]).min(),(X_reduced_tsne[:,2]).max()]

fig = plt.figure(figsize=(8, 6), constrained_layout=True)
ax = fig.add_subplot(111, projection='3d')

# Plotting the data points, coloring them according to the target classes
ax.scatter(X_reduced_tsne[:, 0], X_reduced_tsne[:, 1], X_reduced_tsne[:, 2], c=y_train, cmap=plt.cm.viridis)
ax.view_init(10, -70)
ax.set_xlabel("$x_1$", fontsize=18)
ax.set_ylabel("$x_2$", fontsize=18)
ax.set_zlabel("$x_3$", fontsize=18)
ax.set_xlim(axes[0:2])
ax.set_ylim(axes[2:4])
ax.set_zlim(axes[4:6])

plt.show()
```



2. Use the `digits` dataset to perform a classification task using Linear Support Vector Machine (SVM). Set the parameter `C` to 5. Split the data such that the test split is 25% of the initial dataset. Compare the image classification task using the original number of features (64) to the reduced features (20) using the PCA algorithm. Compare:

- ➢ the training time for both models;
- ➢ the difference in accuracy score.

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from time import time

digits_dataset = datasets.load_digits() # Loading the Digits dataset

# Splitting data into training and testing sets with 25% of data for testing
# 'random_state=0' ensures reproducibility of the split
X_train, X_test, y_train, y_test = train_test_split(digits_dataset['data'],digits_dataset['target'],test_size=0.25,random_state=0)

svm_clf = SVC(kernel="linear", C=5) # Initializing the Support Vector Machine (SVM) classifier with a linear kernel and C=5

# Measuring the time taken to train the SVM on the original data (64 features)
t0 = time() # Start timer
svm_clf.fit(X_train, y_train) # Train the model on the original data

# Calculate and print the training time
t1 = time() - t0
print("Training time with original features (64):", t1)

# Calculating and printing the accuracy score of the SVM on the test set
accuracy_original = svm_clf.score(X_test, y_test)
print("Accuracy with original features (64):", accuracy_original)
```

```
Training time with original features (64): 0.045134544372558594
Accuracy with original features (64): 0.9711111111111111
```

```python
from sklearn.decomposition import PCA

# Applying PCA to reduce dimensions to 20, fitted only on the training set
pca = PCA(n_components=20)
X_train_reduced = pca.fit_transform(X_train)  # Fit PCA on training data
X_test_reduced = pca.transform(X_test)        # Transform test data using the fitted PCA model

svm_clf = SVC(kernel="linear", C=5) # Re-initializing the SVM classifier with the same parameters (linear kernel, C=5)

# Measuring the time taken to train the SVM on the PCA-reduced data (20 features)
t0 = time() # Start timer
svm_clf.fit(X_train_reduced, y_train) # Train the model on the PCA-reduced data

# Calculate and print the training time
t1 = time() - t0
print("Training time with PCA-reduced features (20):", t1)

# Calculating and printing the accuracy score of the SVM on the test set with reduced features
accuracy_reduced = svm_clf.score(X_test_reduced, y_test)
print("Accuracy with PCA-reduced features (20):", accuracy_reduced)
```

```
Training time with PCA-reduced features (20): 0.0383915901184082
Accuracy with PCA-reduced features (20): 0.9688888888888889
```

Thank you !